UniCesumar
Engenharia de Software

Manutenção de Software

Código Limpo – Formatação

- Formatação é importante

- Formatação de código trata de comunicação

- Comunicação é fundamental para desenvolvedores e profissionais

Formatação

- O que podemos fazer para melhorar a formatação de um código?

Dicas de Formatação

- Formatação vertical

- Separação de conceitos

- Funções dependentes

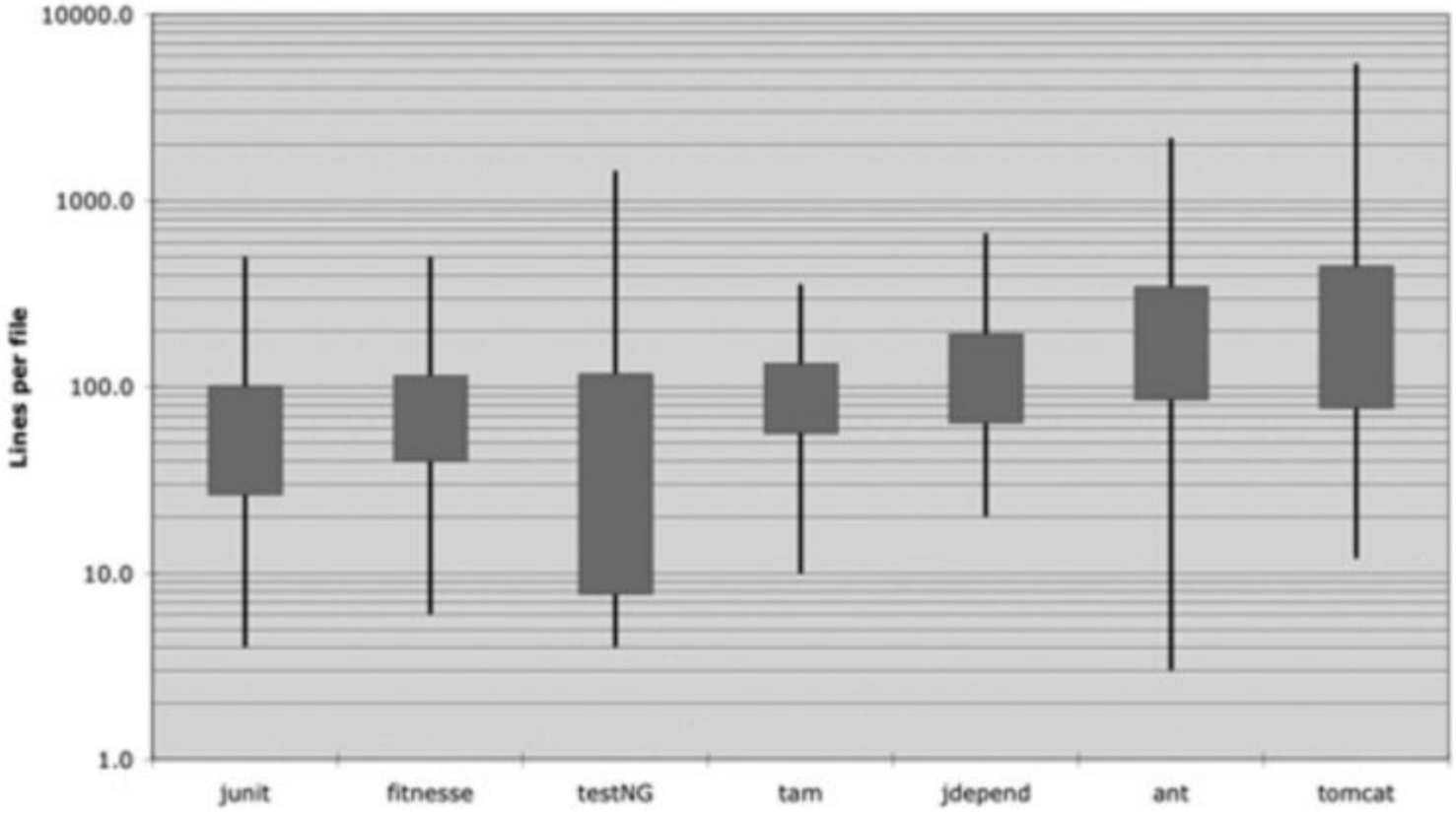
- Afinidade conceitual

- Formatação vertical

- Identação

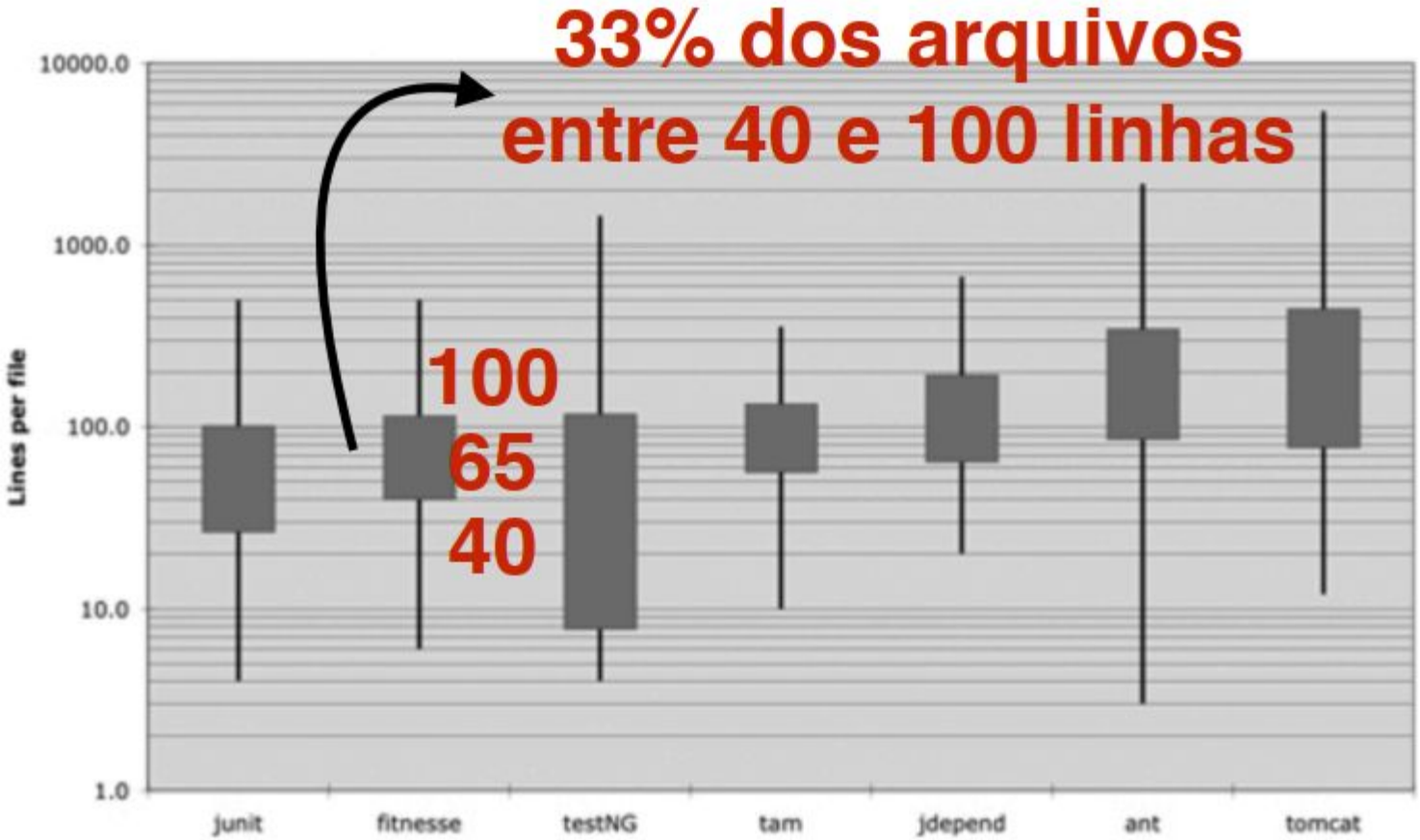
Formatação vertical

- Qual o tamanho um arquivo deve ter?



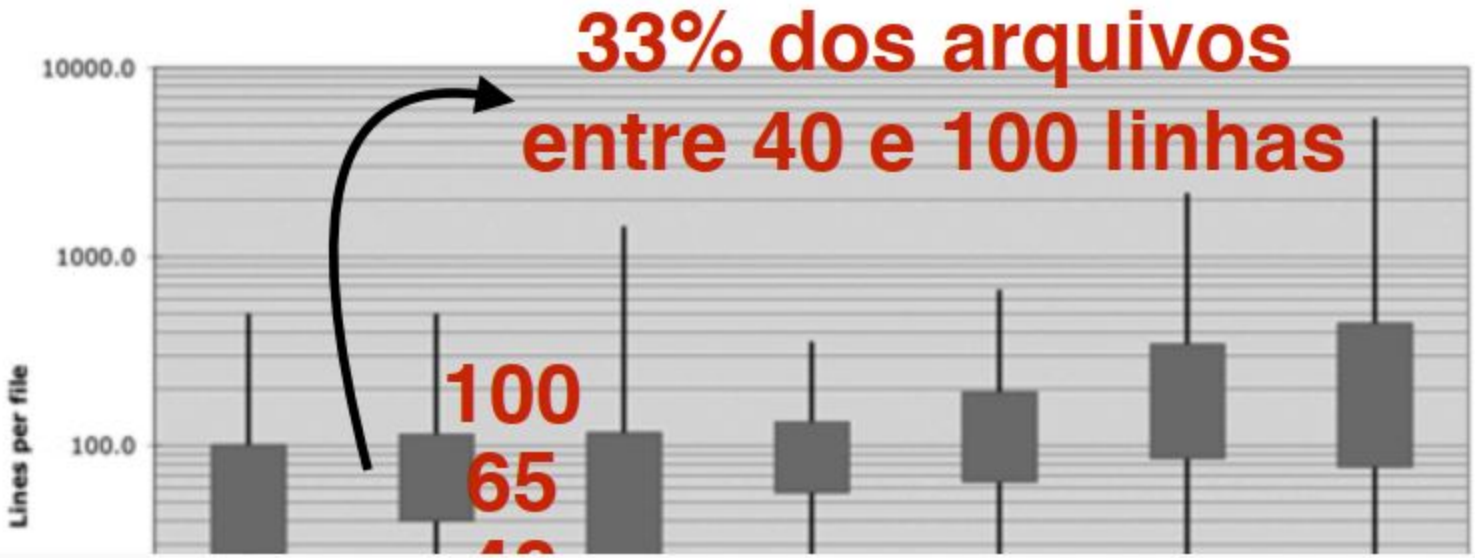
Formatação vertical

- Qual o tamanho um arquivo deve ter?



Formatação vertical

- Qual o tamanho um arquivo deve ter?



É possível construir sistemas reais com arquivos não muito longos

Formatação vertical

- Comparação: código x jornal
- Artigos são lidos verticalmente
- No topo: manchetes
- Primeiro parágrafo: sinopse
- Abaixo: detalhes

R$ 1,50 | Sexta-feira 8 de junho de 2018 | Maringá, PR www.odiario.com | ANO XLIV - NÚMERO 13.527 Esta edição: 18 páginas. Fechada às 21h30

O DIÁRIO DO NORTE DO PARANÁ

clubeDo assinante MUITO+VANTAGENS tudo pensado para você odiario.com/clubedoassinante

ESTATÍSTICAS

Mortes no trânsito têm queda de 43% com radares

JOÃO PAULO SANTOS

● Desde a instalação dos 40 radares, foram registrados **9 óbitos** no trânsito de Maringá, ante 16 no mesmo período do ano passado ● A grande maioria das vítimas fatais são do **sexo masculino**

Lethícia Conegero
lethicia@odiario.com

Desde a instalação dos 40 radares que monitoram 80 faixas em Maringá, em fevereiro deste ano, foram registrados 9 óbitos no trânsito na cidade. O número corresponde ao mês de março, abril e maio, juntos. No mesmo período do ano passado, ocorreram 16 mortes. A redução é de 43,7%.

Tanto no ano passado, como neste ano, a maioria das vítimas fatais eram homens. Durante todo o ano de 2017, 44 pessoas morreram no perímetro urbano de Maringá - com exceção da Avenida Colombo, que é fiscalizada pela Polícia Rodoviária Federal (PRF). Dessas, 39 eram do sexo masculino. Neste ano, de janeiro a maio, foram registrados 15 óbitos, dos quais 11 eram homens.

Segundo o diretor de Fiscalização da Secretaria de Mobilidade Urbana (Semob), Marcelo Filite, o excesso de velocidade é uma das principais causas de acidentes de trânsito. "Na nossa avaliação, o radar está cumprindo o papel dele, que é reduzir os acidentes e óbitos na cidade. A gente percebe a presença de excesso de velocidade nos dois últimos acidentes com óbitos que ocorreram em maio, por exemplo", explica.

Segundo ele, essa diminuição também comprova a eficiência do estudo que foi feito acerca dos pontos com maior índice de acidentes. Mas, foi necessário esperar três meses para constatar a redução no número de óbitos. "Trata-se de um comparativo. Contudo, um veículo com cinco pessoas pode se envolver em um acidente e as cinco pessoas podem morrer. Ou seja, foi um único acidente com cinco óbitos. Por isso, esperamos um período maior para averiguar o real efeito dos radares nesta redução", enfatiza Filite.

Ainda de acordo com o diretor de Fiscalização da Semob, a sociedade está condicionada a só obedecer as regras de trânsito quando há fiscalização. "Na verdade nem precisaria de radar. Deveria ser uma questão de respeito e educação. Mas, quando não há fiscalização, percebemos um aumento de velocidade, pessoas sem cinto, utilizando o celular no volante, etc. E quando é feita a fiscalização, nem que seja por medo, elas deixam de cometer as irregularidades", declara.

A Semob vai analisar as estatísticas de trânsito para planejamento de ações. São verificadas informações sobre local, natureza dos acidentes e outras circunstâncias que levaram essas pessoas a perderem a vida.

O estudo contará com a parceria do Corpo de Bombeiros, das polícias militar, rodoviária estadual e federal, Instituto Médico Legal, Polícia Científica e Samu.

Formatação vertical

- Idealmente um arquivo de código deve ser igual a um artigo de jornal

- Nome: simples e explicativo

- Topo: funções de alto nível

- Mais abaixo: funções com mais detalhes

Separação de conceitos

- Cada grupo de linhas deve representar um conceito
- Regra extremamente simples

```
package fitnesse.wikitext.widgets;
import java.util.regex.*;
public class BoldWidget extends ParentWidget {
  public static final String REGEXP = "'''.+?'''";
  private static final Pattern pattern = Pattern.compile("'''(.+?)'''
    Pattern.MULTILINE + Pattern.DOTALL);
  public BoldWidget(ParentWidget parent, String text) throws Exceptior
    super(parent);
    Matcher match = pattern.matcher(text);
    match.find();
    addChildWidgets(match.group(1));}
  public String render() throws Exception {
    StringBuffer html = new StringBuffer("<b>");
    html.append(childHtml()).append("</b>");
    return html.toString();
  }
}
```

Separação de conceitos

```
package fitnesse.wikitext.widgets;

import java.util.regex.*;

public class BoldWidget extends ParentWidget {
  public static final String REGEXP = "'''.+?'''";
  private static final Pattern pattern = Pattern.compile("'''(.+?)'
    Pattern.MULTILINE + Pattern.DOTALL
  );

  public BoldWidget(ParentWidget parent, String text) throws Except
    super(parent);
    Matcher match = pattern.matcher(text);
    match.find();
    addChildWidgets(match.group(1));
 }

 public String render() throws Exception {
   StringBuffer html = new StringBuffer("<b>");
   html.append(childHtml()).append("</b>");
   return html.toString();
 }

}
```

Funções dependentes

- Se uma função chama outra, elas devem estar próximas umas das outras (chamador acima da chamada)

- Fornece ao código um fluxo natural

```java
public class WikiPageResponder implements SecureResponder {
  protected WikiPage page;
  protected PageData pageData;
  protected String pageTitle;
  protected Request request;
  protected PageCrawler crawler;

  public Response makeResponse(FitNesseContext context, Request request)
    throws Exception {
    String pageName = getPageNameOrDefault(request, "FrontPage");
    loadPage(pageName, context);
    if (page == null)
      return notFoundResponse(context, request);
    else
      return makePageResponse(context);
  }

  private String getPageNameOrDefault(Request request, String defaultPageName)
  {
    String pageName = request.getResource();
    if (StringUtil.isBlank(pageName))
      pageName = defaultPageName;

    return pageName;
  }

  protected void loadPage(String resource, FitNesseContext context)
    throws Exception {
    WikiPagePath path = PathParser.parse(resource);
    crawler = context.root.getPageCrawler();
    crawler.setDeadEndStrategy(new VirtualEnabledPageCrawler());
    page = crawler.getPage(context.root, path);
    if (page != null)
      pageData = page.getData();
  }

  private Response notFoundResponse(FitNesseContext context, Request request)
    throws Exception {
    return new NotFoundResponder().makeResponse(context, request);
  }
```

```
public class WikiPageResponder implements SecureResponder {
  protected WikiPage page;
  protected PageData pageData;
  protected String pageTitle;
  protected Request request;
  protected PageCrawler crawler;

  public Response makeResponse(FitNesseContext context, Request request)
    throws Exception {
    String pageName = getPageNameOrDefault(request, "FrontPage");
    loadPage(pageName, context);
    if (page == null)
      return notFoundResponse(context, request);
    else
      return makePageResponse(context);
  }

  private String getPageNameOrDefault(Request request, String defaultPageName)
  {
    String pageName = request.getResource();
    if (StringUtil.isBlank(pageName))
      pageName = defaultPageName;

    return pageName;
  }

  protected void loadPage(String resource, FitNesseContext context)
    throws Exception {
    WikiPagePath path = PathParser.parse(resource);
    crawler = context.root.getPageCrawler();
    crawler.setDeadEndStrategy(new VirtualEnabledPageCrawler());
    page = crawler.getPage(context.root, path);
    if (page != null)
      pageData = page.getData();
  }

  private Response notFoundResponse(FitNesseContext context, Request request)
    throws Exception {
    return new NotFoundResponder().makeResponse(context, request);
  }
```

1 2 3

Afinidade conceitual

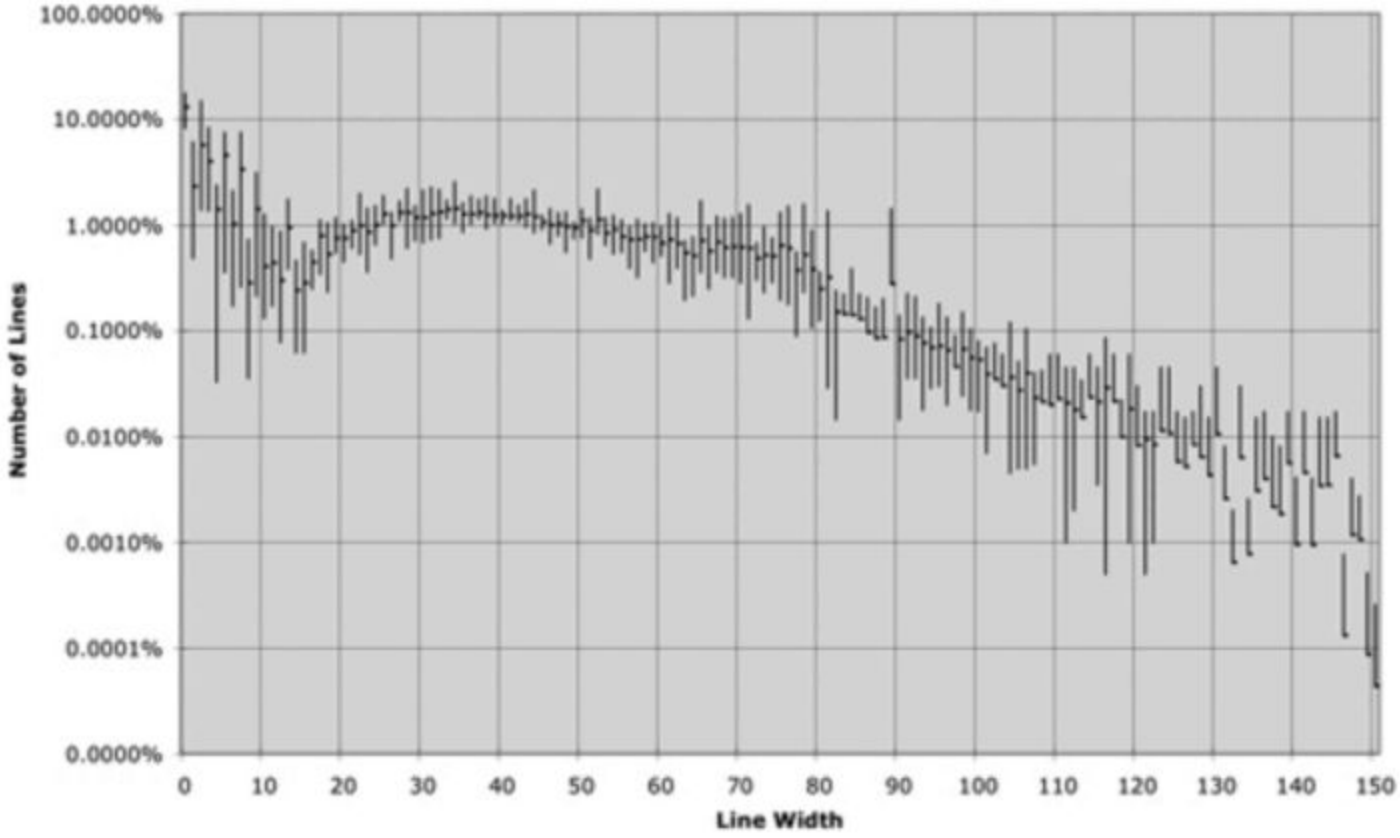
- Dependência direta ou operações similares

Variações da mesma tarefa básica

```
public class Assert {
  static public void assertTrue(String message, boolean condition) {
    if (!condition)
      fail(message);
  }

  static public void assertTrue(boolean condition) {
    assertTrue(null, condition);
  }

  static public void assertFalse(String message, boolean condition) {
    assertTrue(message, !condition);
  }

  static public void assertFalse(boolean condition) {
    assertFalse(null, condition);
  }
…
```

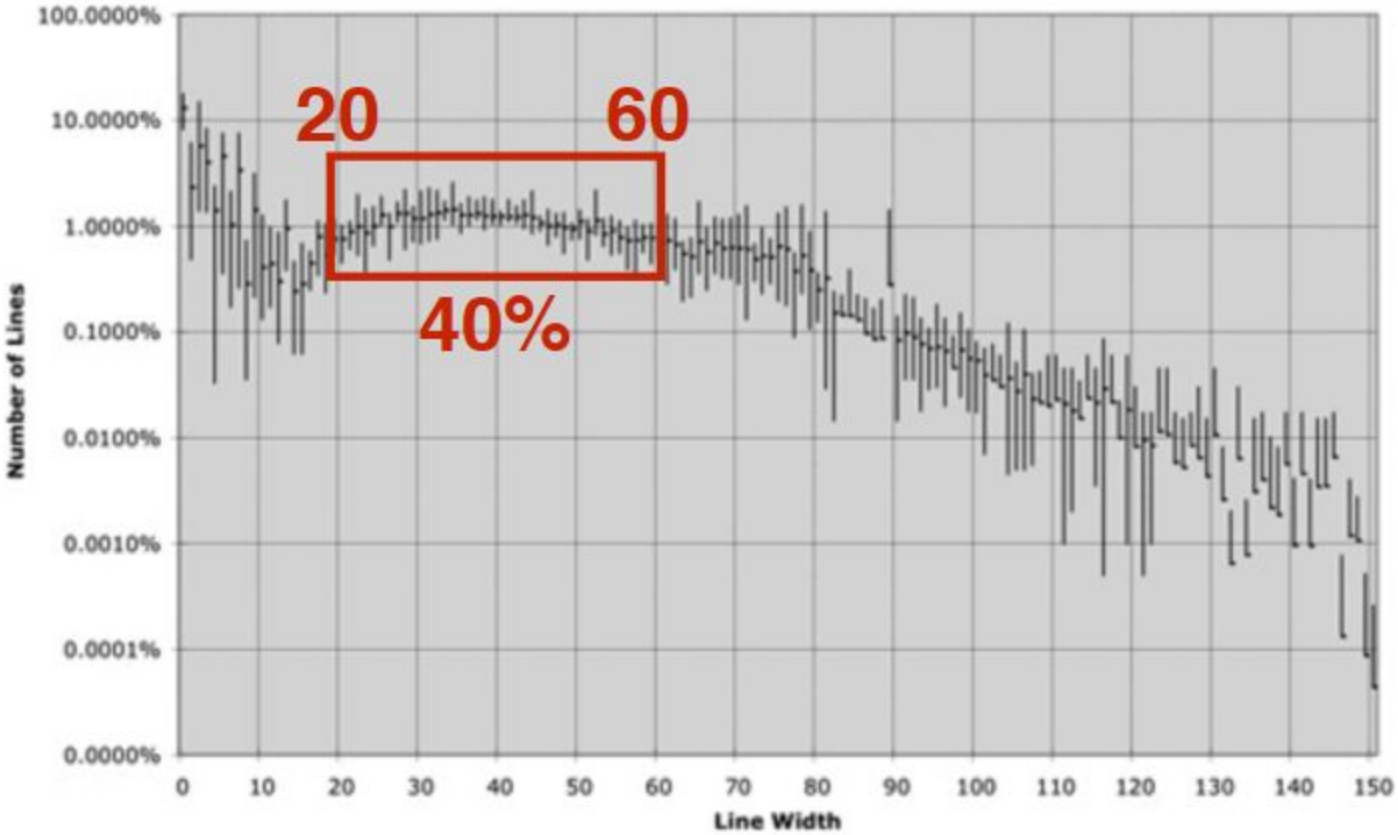
Formatação horizontal

- O quão comprida uma linha deve ser?



Formatação horizontal

- O quão comprida uma linha deve ser?



Formatação horizontal

- O quão comprida uma linha deve ser?

Identação

- Código fonte é hierárquico
  - Classes, métodos, blocos de código, etc
  - Cada nível é um escopo, onde variáveis podem ser declaradas e utilizadas

- Identação: torna os níveis visíveis
  - Permite visualizar facilmente os escopos dos ifs, whiles, fors

- Sem identação: código tornam-se ilegíveis para humanos

```
public class FitNesseServer implements SocketServer { private FitNesseCont
context; public FitNesseServer(FitNesseContext context) { this.context =
context; } public void serve(Socket s) { serve(s, 10000); } public void
serve(Socket s, long requestTimeout) { try { FitNesseExpediter sender = ne
FitNesseExpediter(s, context);
sender.setRequestParsingTimeLimit(requestTimeout); sender.start(); }
catch(Exception e) { e.printStackTrace(); } } }
```

```
public class FitNesseServer implements SocketServer { private FitNesseCont
context; public FitNesseServer(FitNesseContext context) { this.context =
context; } public void serve(Socket s) { serve(s, 10000); } public void
serve(Socket s, long requestTimeout) { try { FitNesseExpediter sender = ne
FitNesseExpediter(s, context);
sender.setRequestParsingTimeLimit(requestTimeout); sender.start(); }
catch(Exception e) { e.printStackTrace(); } } }
```

```
public class FitNesseServer implements SocketServer {
  private FitNesseContext context;


public FitNesseServer(FitNesseContext context) {
  this.context = context;
}

public void serve(Socket s) {
  serve(s, 10000);
}

public void serve(Socket s, long requestTimeout) {
  try {
    FitNesseExpediter sender = new FitNesseExpediter(s, context);
    sender.setRequestParsingTimeLimit(requestTimeout);
    sender.start();
  }
  catch (Exception e) {
    e.printStackTrace();
  }
}
```